

路线 6: Hybrid MILP / MIQP 分解与量子子问题

Hybrid 路线的核心态度很务实：不要把完整混合整数问题硬塞进量子求解器。先拆问题，把经典方法擅长的部分交给经典，把小而关键的离散子问题交给 QUBO、退火或 QAOA。

这条路线通常是比赛里最稳的工程主线。

MILP / MIQP 是什么

MILP 是 Mixed Integer Linear Programming:

```
linear objective
linear constraints
some variables integer or binary
some variables continuous
```

MIQP 是 Mixed Integer Quadratic Programming:

```
quadratic objective
linear or quadratic structure
mixed binary / integer / continuous variables
```

很多真实题都不是纯 binary:

- 电力：机组开关是 binary，发电量是 continuous。
- 物流：是否走边是 binary，到达时间可能是 continuous。
- 投资组合：是否选资产是 binary，持仓比例可能是 continuous。
- 排产：机器选择是 discrete，时间变量可能有连续或大整数结构。

如果强行把所有变量都 QUBO 化，很容易变量爆炸。

Hybrid 的一句话

经典方法管全局结构和连续部分，量子/量子启发方法管小规模离散搜索。

这不是退让，而是工程上更成熟的用法。

Relax-Round-Repair

最常见的 hybrid pipeline 是：

1. Relax
2. Round
3. Repair
4. Polish

Relax

把 binary/integer 变量放松成连续变量。例如：

$$x \in \{0, 1\} \rightarrow 0 \leq x \leq 1$$

这样可以用 LP/QP 求一个容易得到的松弛解。

Round

把连续松弛解变回离散解。例如 $x = 0.91$ round 成 1, $x = 0.12$ round 成 0。

Repair

round 后可能违反约束，需要修复。

Polish

固定离散变量后，重新优化连续变量或做局部搜索，让解更好。

Fix-and-Optimize

另一个很实用的 hybrid 方法是 fix-and-optimize:

1. 先有一个 `incumbent` 解
2. 固定大部分变量
3. 只放开一小组变量
4. 对这组变量构造 QUBO 子问题
5. 用退火 / QAOA / local search 求改进
6. 接受更好的解
7. 换下一组变量重复

这很适合量子模块，因为子问题规模可控。

你可以把它理解成：完整问题太大，量子模块每次只负责一个局部改良窗口。

ADMM 分解直觉

ADMM 是一种分块优化思路。它把问题拆成多个 block，每个 block 解自己的子问题，再通过惩罚项和一致性变量协调。

在 mixed-binary optimization 里，可以把连续块和离散块分开：

```
continuous block -> classical optimizer
binary block      -> QUBO / Ising solver
coordination      -> penalty / dual update
```

第一遍学习 ADMM 不必深挖推导，先记它的用途：

当一个问题天然分块时，ADMM 可以让量子子问题嵌入迭代优化流程。

Warm-start 和 confidence

Hybrid 路线很适合给量子模块 warm-start。

连续松弛解里，变量值接近 0 或 1，说明它比较确定：

```
x_i = 0.98 -> 很可能取 1
x_j = 0.03 -> 很可能取 0
x_k = 0.51 -> 不确定
```

一种策略是：

固定高置信度变量，只把不确定变量交给 QUBO 子问题。

这样既缩小了量子问题规模，又保留了关键搜索空间。

Incumbent 和 bounds

Hybrid 方法必须记录两个概念：

Incumbent

当前找到的最好可行解。

Bound

松弛问题或经典 solver 给出的理论界。它帮助你估计最优性差距。

比赛里如果能展示：

```
当前解 objective = 123
relaxation bound = 118
gap = 4.2%
```

评委会更容易相信你的方案不是只会吐一个数字。

一个小例子

假设有一个模型部署问题：

- binary：选哪个模型、开哪些加速选项
- continuous：给每个模型分多少资源
- constraints：预算、延迟、吞吐

直接全量 QUBO 可能很重。

Hybrid 可以这样做：

1. 连续松弛求一个近似资源分配
2. 根据松弛解判断哪些 `binary` 变量确定
3. 对不确定 `binary` 变量构造小 QUBO
4. 用 SA 或 QAOA 求子问题
5. 固定 `binary` 后重新优化 `continuous resource`
6. `repair` 预算或延迟约束

这比“所有变量一次性 QUBO”稳得多。

比赛时能改哪些地方

分解策略

哪些变量固定，哪些进入子问题，是核心算法设计点。

子问题大小

量子模块适合小规模。你可以控制每轮放开的变量数。

Rounding 策略

简单 threshold、随机 rounding、constraint-aware rounding、按收益排序 rounding 都可以比较。

Repair 策略

Hybrid 的 repair 可以用贪心、局部搜索、经典 solver。这里很容易做出实际效果。

子问题 solver

同一个 QUBO 子问题可以用 exact、SA、QAOA、D-Wave 比较。

迭代规则

什么时候接受新解，什么时候扩大子问题，什么时候停止，都能设计。

常见坑

把量子模块说成全局 solver。

Hybrid 里量子模块通常是 discrete oracle 或 local improver。要讲清楚职责。

松弛解太乐观。

Relaxation bound 可能离可行整数解很远。不要把松弛解当最终解。

Round 后不 repair。

Round 出来的整数解很可能违反约束。

子问题没有上下文。

固定变量后，子问题目标和约束要包含被固定变量带来的影响。

没有 benchmark。

Hybrid 方案必须和纯 classical baseline、纯 QUBO/SA、random/greedy 做对比。

和其他路线的关系

路线 1 提供 QUBO 子问题表达。

路线 2 提供子问题约束处理和 repair。

路线 3、4、5 都可以作为 hybrid 子问题求解器。

路线 6 决定整体工程架构：什么时候经典求，什么时候量子求，什么时候修复和 polish。

共读任务

拿一个大问题问三句话：

1. 哪些变量是连续的，哪些是 binary/integer？
2. 哪些变量很确定，可以先固定？
3. 哪一小块变量最值得交给 QUBO/量子模块搜索？

如果你能回答这三句，说明你已经开始从“算法名字”转向“系统设计”。

延伸阅读

- `literature/requirements/06_hybrid_milp_miqp_requirements.md`
- Braine et al., mixed binary optimization with quantum subroutines
- Gambella & Simonetto, multi-block ADMM for mixed binary optimization
- Brown et al., mixed binary quadratic optimization and Ising solvers